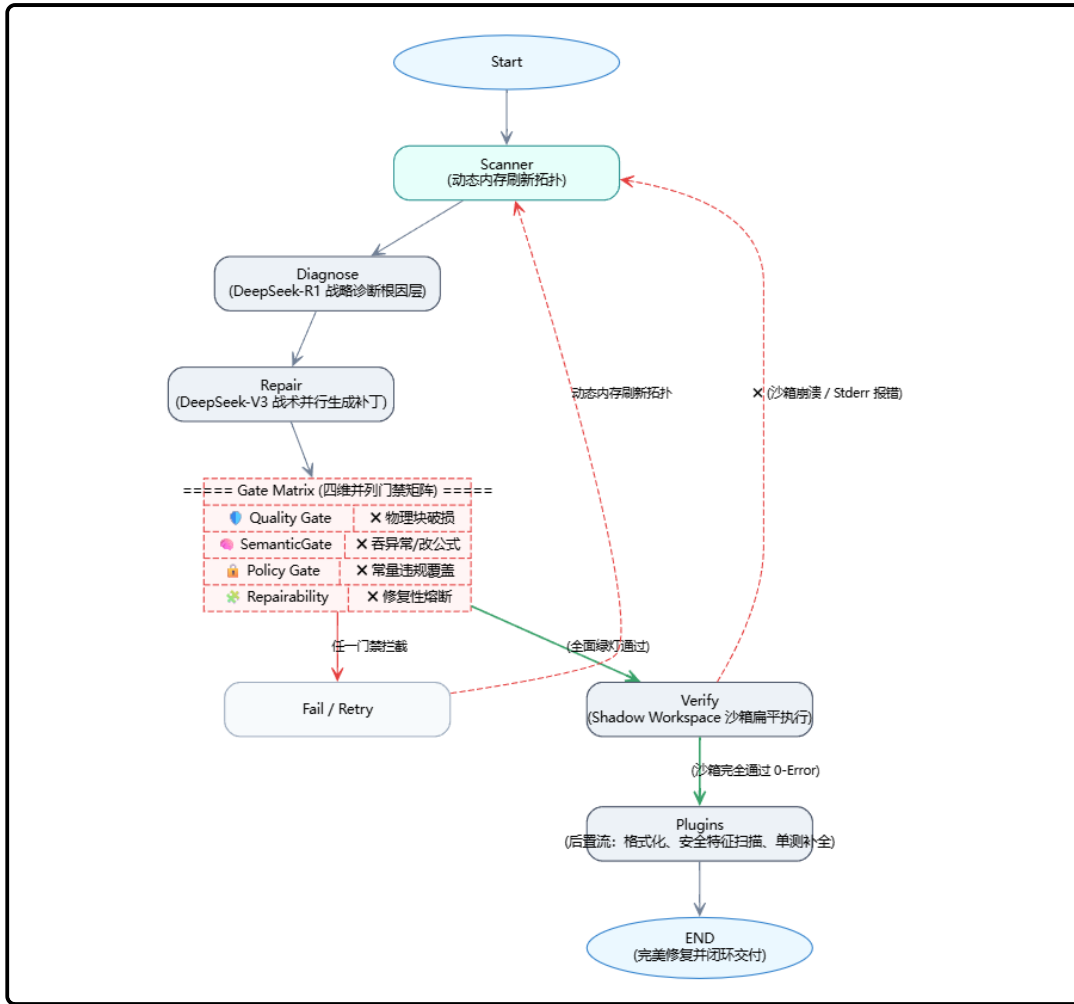


🔗 LLM-Code-Medic: 工业级多文件智能协同修复系统工作原理技术报告

本报告全面剖析了一个基于大语言模型双擎驱动、全域抽象语法树（AST）感知、具备四维工程级并列质量门禁与安全影子沙箱的自主代码修复智能体系统。系统攻克了工业级大型复杂软件在无监督/半监督环境下的自动定位、依赖对齐、合规审查与行为退化阻断等核心技术痛点。

1. 任务分解与编排机制（Task Decomposition & Orchestration）

智能体的核心调度中枢采用“数据流驱动的有状态分布式拓扑机”（Stateful Distributed Topology Engine）。系统拒绝使用传统的一条龙式、不可逆的线性 Prompt 结构，而是将修复任务解构成一个具备深度自反思与动态刷新能力的环形闭环网络。



核心节点分布式流转原理:

Figure 1

- diagnose_node (战略诊断节点)**: 此节点作为智能体的“司令部”。它在接收到异常信号（如测试失败、线上崩溃堆栈）后，首先调用静态感知层对当前受损仓库进行全量索引，随后构建极高长文本密度的战略上下文。其核心职责是生成确定性的故障清单（BUG_INVENTORY），锁定修复边界。
- repair_node (战术补丁生成节点)**: 此节点是智能体的“兵工厂”。它承接 `diagnose_node` 下达的范围约束（`target_files`）以及上一轮门禁或沙箱反馈的对抗性报错日志。其职责是高效、精确地编写原子化代码补丁（Patch），并将其动态合并入内存态的代码字典中。

- `Gate Matrix` **（四维并列门禁矩阵）**：补丁合并完成后，不经过编译，而是直接并列分发至四大门禁进行代码层面的静态和逻辑合规检查。
- `verify_node` **（沙箱验证节点）**：全面绿灯通过门禁后，代码被刷写进磁盘的影子工作区，启动物理隔离的多维动态行为审计。一旦执行报错（`returncode != 0` 或 `stderr` 含有异常），异常指纹将原路返回至 `Scanner` 重新激活拓扑刷新，驱动状态机进入下一轮纠偏迭代。

2. 异构大模型选择与弹性降级流控（LLM Routing）

智能体架构在工业落地的关键在于平衡**逻辑推理深度**、**吞吐延时**、**API 成本与系统单点故障风险**。系统抛弃了单一模型走天下的做法，构建了基于异构大模型的双擎驱动与跨供应商弹性切流拓扑。

2.1 战略与战术双擎选择

- **战略根因层选用 DeepSeek-R1**：长链条推理、自我反思与强化学习衍生出的思考能力是 R1 的绝对优势。在诊断阶段，系统需要跨越复杂的报错堆栈，溯源多个模块之间的隐式业务逻辑违规。R1 的 DeepSeek-R1-Thinking 过程能完美解构这种隐式冲突，确保输出的 `BUG_INVENTORY` 具备 100% 的高置信度。
- **战术生成层选用 DeepSeek-V3**：在获得 R1 明确的战略指导（改哪里、怎么改、约束是什么）后，具体的代码编写属于高吞吐的确定性事务。DeepSeek-V3 拥有极高的代码生成速度、优异的指令遵循能力与极低的 Token 成本。利用 V3 进行并行补丁生成，可大幅收敛智能体的整体响应延时。

2.2 跨供应商对角线弹性降级流控算法

在生产环境中，网络抖动、高并发导致的 `429 Rate Limit` 或供应商服务中断（如 `503 Service Unavailable`）是不可避免的。智能体通过内核中的 `ModelFactory`、`ProviderRouter` 和 `LLMInvoker` 实现了无缝切流：

- 1. 重试控制权绝对收回**：系统在封装底层大模型 SDK 时，强制指定底层客户端的 `max_retries=0`。此举是为了防止底层 SDK 进行无感知的盲目阻塞，将有状态的退避重试权（Exponential Backoff）完全收回至智能体调度框架。
- 2. 异构漂移协议（Diagonal Cutover）**：当主模型发生异常时，`LLMInvoker` 会启动重试机制。一旦重试耗尽且配置允许降级（`allow_fallback=True`），系统调用路由器实施对角线对冲漂移：
 - 若 **DeepSeek** 服务链路崩溃 → 动态切流至 **Gemini** 平台（如 `gemini-2.5-pro` 承接诊断，`gemini-2.5-flash` 承接修复）。
 - 若 **Gemini** 平台发生不可抗力熔断 → 自动回退切流至 **DeepSeek** 专属隔离网关。

3. 深度提示工程策略与伪代码断言约束（Prompt Engineering）

智能体拒绝使用任何具有模糊、宽泛特性的自然语言 Prompt，而是发展出了一套“基于伪代码断言的高强度约束规约语言”，以此锁死大模型的发散行为。

根据提供的工业级 `prompts.py` 源代码，我为您将文本报告中的**10个工程合规断言问题（Q1-Q10）**、**十大禁止模式（Forbidden Patterns）**以及***诊断与状态偏转逻辑**进行了完整的对齐与补充。

以下是补充完整后的分析报告：

3.1 DIAGNOSE_SYSTEM_PROMPT（战略诊断高压线）

诊断 Prompt 的核心在于强迫大模型 像静态编译器一样工作，严禁跳步。它包含以下硬性审计步骤与状态偏转逻辑：

3.1.1 [DIAGNOSE AUDIT PROFILE]

- **STEP A: Module Exists Check** → 核实发生故障的物理文件路径（`import X` 或 `from X import Y` 中的模块 X）是否真实存在于仓库中。若不存在，触发 `BUG [MODULE_NOT_FOUND: X]`。
- **STEP B: Symbol Exists Check** → 解析目标语法树，核实类、函数及变量定义。检查 `from X import Y` 中的 Y 是否在 X 的顶级定义列表中。若不存在，触发 `BUG [SYMBOL_NOT_FOUND: Y in X] → CORRECT: <actual name>`。此步骤对全域 `import` 强制执行。
- **STEP C: Call Site Signature Check** → 比对 Caller（调用方）与 Callee（被调用方）的形参及实参数量/类型。若不匹配，触发 `BUG [SIGNATURE_MISMATCH: f]`。
- **STEP D: Argument Provenance Check** → 追踪传参溯源，审查是否存在非法动态污染或硬编码硬轨。若发现硬编码字面量且仓库存在同义常量，触发 `BUG [HARDCODED_LITERAL: use <MODULE>.<CONSTANT_NAME> instead]`。
- **STEP E: Batching** → 强制汇总生成唯一种子 `BUG_INVENTORY`，严禁执行瀑布式局部修复。

3.1.2 状态偏转与死循环击碎逻辑

诊断 Prompt 中嵌入了有状态的循环断言（`LOOP_CHECK`）。

- **触发条件：** 当智能体检测到全局沙箱报错非空（`sandbox_stderr is non-empty`）且修复尝试轮次 `repair_attempts >= 1`。
- **三项核心断言（LOOP_CHECK_1 到 3）：**
 1. 当前报错的异常类型是否与前几轮高度雷同（`Same exception type`）？
 2. 前一轮的修复动作是否仅仅局限在被调用方（Callee）文件内部（`Only modified callee files`）？
 3. 被调用方（Callee）内部在经历上次修复后，是否已经包含了明确的 `raise` 防御断言（`Already contains a raise`）？
- **状态偏转结果：** 若上述三项断言全部为 **YES**，Prompt 将强制封杀 `[CONTRACT_UNDEFINED]`（契约未定义）分类路线，将 `LOOP_VERDICT` 判定为 `[CALLER_VIOLATED_CONFIRMED]`，并强行将模型认知锁定为 `[CALLER_VIOLATED]`（调用侧违规），同时强制要求修复范围（`REPAIR_SCOPE`）必须包含调用方文件，**严禁任何针对 Callee 的单边修改**。

这在工程上具有决定性意义：它彻底打破了 AI 在面对报错时，只会拼命在 Callee（被调用方）内部疯狂打补丁、吞异常、加默认值的“思维泥潭”，逼迫 AI 转向全局，去重构和修正上游调用侧的入参。

3.2 REPAIR_SYSTEM_PROMPT（战术生成硬红线）

修复 Prompt 制定了严厉的**十大禁止模式（Forbidden Patterns）**，属于一票否决制的安全红线。任何修复代码只要触发任意一条模式，将直接作废并触发重试（RESTART）：

3.2.1 十大禁止模式（Forbidden Patterns）

1. **Symptom Masking（掩盖症状）：** 严禁通过编写调用侧防御性检查、早期返回、静默回退或异常包装，在保留原始非法状态/非法值的前提下消除异常。引发非法值的源头函数（Producer）才是唯一的修复目标。
2. **Exception Swallowing（吞噬异常）：** 严禁编写 `try/except` 块且捕获后不进行重新抛出（`re-raise`）或升级抛出（`escalate`）。拒绝任何形式的“优雅降级”，必须保持“快错（Fail-fast）”契约。

3. **Magic Number Injection（凭空注入硬编码幻数）**：严禁注入任何未经业务文档支撑的硬编码数字常量。基于非法路径返回硬编码默认值被视作“数据伪造”，一律拒绝。
4. **Shim with Undocumented Default（未定义默认值的未授权垫片）**：严禁通过 `def old_api(x, weight=<hardcoded>)` 等形式桥接新老接口，除非满足两个硬性前置：a) 拥有历史文档记载的默认值；b) 显式抛出 `DeprecationWarning`。否则必须在垫片内部抛出 `NotImplementedError`。
5. **Formula / Threshold Mutation（业务公式/阈值篡改）**：严禁通过微调业务公式（如将 `weight - 10` 改为 `weight - 9`）或修改算法阈值来迎合单次测试。业务公式属于不可变状态（Business Invariant），必须修复输入源，而非篡改公式本身。
6. **Callee-Only Loop Repair（死循环单边修复）**：当 `LOOP_VERDICT == [CALLER_VIOLATED_CONFIRMED]` 时，严禁仅修改被调用方（Callee）文件或变动其 Guard 拦截面。
7. **Invented Caller Value（凭空捏造调用值）**：严禁在无法从当前代码仓库上下文（常量、Docstring、其他位置的统一用法）中推导出有效值的情况下，主观猜测并修改调用侧入参。若无法证明值来源，必须输出 `ESCALATE_REQUIRED`。
8. **Invented Import Symbol（幻觉捏造未声明符号）**：严禁通过 `from X import Y` 捏造未在 X 源码中真实声明的符号。必须完全依据 `BUG_INVENTORY` 中核实的 `CORRECT` 字段填入。
9. **Hardcoded Literal Replacing a Repo Constant（硬编码字面量替代仓库常量）**：当仓库上下文已经提供了命名常量（如全大写 `ALL_CAPS` 或特定前缀的模块级变量）时，严禁直接在代码中写入字符串或路径字面量。
10. **Partial BUG_INVENTORY Repair（局部/瀑布式修复）**：严禁单轮只修复部分缺陷。每一项被收录在 `BUG_INVENTORY` 中的缺陷都必须在当前轮次彻底解决，瀑布式分步修复属于严重违规。

3.2.2 自校验（Self-Verification）Q1-Q10 问题列表

为了实现自校验（Self-Verification），修复 Prompt 强制要求 DeepSeek-V3 在输出最终的代码块 `<<<FILE_PATH>>>` 之前，必须在内部思考区依次完整回答 10 个工程合规断言问题（Q1-Q10），并提供严密的分类对齐证据：

- **Q1:** 本修复补丁是否准确解决（Address）了诊断报告中指出的根因层（`root cause layer`）？
- **Q2:** 我是否在没有定义命名变量和加注业务注释的情况下，凭空引入了任何数字常量？（**预警：若 YES 则重启**）
- **Q3:** 修复代码中是否存在任何捕获了异常（`catch block`）却未能重新抛出（`re-raise`）或升级抛出（`escalate`）的情况？（**预警：若 YES 则重启**）
- **Q4:** 我是否在缺少历史文档明确记载默认值的前提下，擅自添加了接口垫片（`shim`）？（**预警：若 YES 则重启**）
- **Q5:** 我是否私自篡改、微调了任何业务计算公式、既定阈值或算术常量？（**预警：若 YES 则重启**）
- **Q6:** 如果当前状态 `LOOP_VERDICT == [CALLER_VIOLATED_CONFIRMED]`，我是否严格避免了修改被调用方（Callee）的 Guard 防御屏障？
- **Q7:** 每一个被修改的代码行，其修改依据是否均能完全对齐 `ROOT_CAUSE_CLASS` 或 `LOOP_VERDICT` 的诊断结论？
- **Q8:** 所有变动的或新引入的 `import` 符号，是否完全匹配 `BUG_INVENTORY` 中的 `CORRECT` 字段或经过源码实证？
- **Q9:** 在仓库上下文已经定义了对应命名常量的前提下，我是否依然写入了硬编码的字面量？（**预警：若 YES 则重启**）
- **Q10:** 本修复是否百分之百覆盖并解决了 `BUG_INVENTORY` 中的每一条缺陷记录？（必须逐一列出条目并予以确认）

3.2.3 自动化硬重启（RESTART）触发红线：

只有当 10 个问题的断言全部符合以下逻辑矩阵时，模型才被允许输出补丁，从源头上遏制了幻觉的扩散：

RESTART if: $Q1 = \text{NO} \mid Q2 = \text{YES} \mid Q3 = \text{YES} \mid Q4 = \text{YES} \mid Q5 = \text{YES} \mid Q6 = \text{NO} \mid Q7 = \text{NO}$

4. 工具调用与全域静态分析机制（Tool Calling & AST Parsing）

智能体之所以具备高置信度，是因为其拥有不依赖代码实际执行的“全域静态代码感知与影响域推导”工具链。这主要由 `ProjectScanner` 与 `AST Resolver` 协同实现。

4.1 符号指纹提取技术

`ProjectScanner` 通过 Python 原生 `ast.parse()` 解析所有关联源文件，直接提取三层图谱：

- 全局符号导出表（Export Table）：遍历 `ast.ClassDef` 与 `ast.FunctionDef` 节点，下钻提取类体内的所有实例方法和类方法，剔除私有魔术方法，构筑出项目的功能矩阵。
- 导入依赖图（Import Graph）：捕获 `ast.Import` 与 `ast.ImportFrom` 节点，将点号路径（如 `a.b.c`）翻译为规范化的物理路径（`a/b/c.py`），编织出文件树拓扑。
- 方法调用图（Call Graph）：深度遍历全语法树中的 `ast.Call`（调用）节点，抓取直调或属性调用标识符，构建全域调用热力图。

4.2 影响域前向闭包追溯算法

当诊断节点锁定受损文件集合 T_0 时，`ast_resolver.py` 中的 `expand_target_files` 启动基于图论的前向闭包无限迭代算法。它不仅审阅受损文件本身，还通过以下递推公式向全域发散：

$$\text{Result}_{t+1} = \text{Result}_t \cup \{f \mid \text{ImportGraph}[f] \cap \text{Result}_t \neq \emptyset\} \cup \{c \mid \text{CallGraph}[c] \cap \text{Result}_t \neq \emptyset\}$$

算法持续运行，直到集合不再发生任何扩张。

工具价值：此项工具调用能确保所有直接或间接调用了该故障文件的上游 Caller、以及被其引用的下游下游依赖，全部无遗漏地合并进 LLM 的审阅上下文，彻底消除了 AI 因“一叶障目”而引发的下游重构坍塌。

5. 四维工程级质量门禁矩阵（Gate Matrix）

智能体最坚固的防御屏障在于其并列执行的四大质量门禁矩阵。补丁生成后，必须同时、并列横穿这四道关卡，任何一关抛出违规，立即熔断当前补丁流，退回上游进行 Fail-Retry 重新迭代。

5.1 四维门禁并列审查矩阵

门禁组件名称	技术实现原理	核心防御目标	处置拦截手段
PatchQualityGate (基本质量门)	实测代码字符剪裁比对、正则文件指纹映射	防止 AI 伪造空文件、误抹除核心逻辑、扩散至非受控无关文件	一票否决制，直接熔断并判定补丁失效

Table 1

门禁组件名称	技术实现原理	核心防御目标	处置拦截手段
PolicyGate（合规策略门）	<code>ast.literal_eval</code> 常量定值提取、前/后快照键值对比	严禁 AI 私自调整受控业务变量、阈值或重试计数等全局敏感参数	触发 <code>CALLER_INPUT_MUTATION</code> 报警并终止
SemanticPatchGate（语义特征门）	深度 AST 节点类型断言遍历、函数签名长度分析、公式常量完整性审计	阻断恶性代码变异（如除零分支造假、隐式吞异常、计算公式篡改等）	抛出规则违反详情（Rule 0-7），发回重试轮次
RepairabilityGate（可修复性评估门）	动态滑动窗口特征捕获、重复异常指纹分析	识别死循环、思维盲区，及时熔断自动化并唤醒人机协作	触发自动化流程熔断，偏转至三模态人工协同

5.2

SemanticPatchGate 核心规则的 AST 原理深剖

SemanticPatchGate 通过在抽象语法树上挂载多种静态分析特征匹配器，对生成的代码进行底层语义透视：

- **Rule-1 (数据伪造审计)：**AI 经常会在遇到除零异常（`ZeroDivisionError`）时，写出硬编码修复。门禁会审查所有包含 `ast.If` 且条件中含有 `== 0` 比较的除数防御分支。一旦发现该分支内的执行体是直接 `return Constant`（返回一个捏造的假数字来欺骗外部调用，如发生除零直接返回 0 或 -1），而不是 `raise ValueError`，判定为恶意伪造，**拦截！**
- **Rule-2 (隐式吞异常审计)：**遍历所有的 `ast.ExceptHandler` 节点。若发现其 `body` 块的长度为 1，且节点类型为 `ast.Pass` 或值为 `Ellipsis`（即 `...`），判定 AI 试图通过空 `except` 块静默抹除线上崩溃，**拦截！**
- **Rule-4 (核心计算公式保护)：**这是系统最精密的数学防线。门禁首先提取模块作用域内的隐藏及大写具名常量字典。然后，它对旧函数和新函数内部涉及二进制算术运算（`ast.BinOp`）的所有数字字面量、具名常量进行全解析比对。若旧函数运算中的任何基准数值在新函数的解析结果中丢失或改变（例如将算法公式中的 $\times 1.59$ 偷偷改为了 $\times 2.0$ ），判定核心算法逻辑遭非法篡改，**拦截！**（注：将字面量提取为具名常量是允许的，但数值绝对值不能变）。
- **Rule-5 (签名漂移审计)：**比对新旧函数的参数声明列表 `args.args` 长度。若长度变更（如修改了核心公共 API 签名），直接触发签名漂移保护，**拦截！**

5.3

RepairabilityGate 人机协同机制与上下文编译

作为并列的第四道大门，`RepairabilityGate` 负责动态追踪智能体的精神状态。它持续维护一个长度为 5 的 `error_signature_history` 堆栈，从沙箱崩溃的 `stderr` 中动态抽取 `错误类型@文件路径` 组成的特征指纹（例如 `attributeerror@utils/helpers.py`）。

一旦发现连续 3 轮的异常指纹完全雷同，或 R1 明确要求 `ESCALATE_REQUIRED`，门禁判定智能体已陷入思维死循环，立即熔断状态机，转入人机协同交互三模态，并对后续的 Prompt 上下文进行定制化动态编译：

1. **GUIDED（业务引导模式）**：工程师输入核心业务解释。系统基于输入动态编译出【用户授权业务上下文 - GUIDED 模式】。大模型临时获权允许微调 Caller 参数，但修正值必须绝对与工程师的说明对齐，严禁凭空发明业务数字。
2. **OVERRIDE（特权例外模式）**：针对历史遗留代码导致的兼容死锁，工程师选择此模式。系统编译出特权上下文，获权允许引入不带 Deprecation 的兼容 Shim 垫片代码。但触发强制红线约束：**大模型必须在新代码的上方附加一字不差的硬编码注释** `# OVERRIDE: <工程师输入的授权理由>`，否则将永远无法通过后续的 Git CI 审计。
3. **STRICT（标准终止模式）**：完全终止自动化，优雅导出当前的静态分析拓扑数据与根因分类报告，由人工手动接管修复。

6. 沙箱安全隔离与双向行为回归审计（Sandbox Verification）

通过门禁矩阵全量绿灯的代码，将合入位于独立的物理路径 `output/` 下的影子工作区（Shadow Workspace）。

6.1 影子空间路径穿越防御（Security Guardrail）

沙箱组件 `CodeExecutor` 通过全量复制物理镜像来构建无状态工作区。在写入任何 Patch 文件前，系统调用 `safe_join` 算法对抗潜在的恶意代码注入：

Python

```
1 def safe_join(repo_root: str, relative_path: str) -> str:
2     root_real = os.path.realpath(repo_root)
3     full_real = os.path.realpath(os.path.join(repo_root, relative_path))
4     if os.path.commonpath([root_real, full_real]) != root_real:
5         raise ValueError(f"检测到非法路径穿越攻击: {relative_path}")
6     return full_real
```

Fence 1

一旦大模型因对抗性漏洞在生成的路径中夹带了 `../../etc/passwd` 等穿越指纹，系统立即强行中止，确保宿主系统处于绝对安全的隔离区。同时，沙箱环境强行注入 `PYTHONDONTWRITEBYTECODE=1`，彻底切断多次测试之间的遗留缓存污染。

6.2 数据驱动型控制台行为回归分析

沙箱不仅通过监测 `returncode == 0` 来验证“代码不崩溃”，还引入了独创的**动态数据指纹回归模型**。系统会动态抓取 Baseline 代码与 Repaired 代码在控制台（`stdout`）输出的最后一段文本内容，利用正则表达式解析出核心的关键业务数字（原数字设为 $V_{baseline}$ ，新数字设为 $V_{repaired}$ ）。

系统计算其变异比例系数：

$$ratio = \frac{|V_{repaired}|}{\max(|V_{baseline}|, 10^{-8})}$$

- **变异拦截断言**：一旦发现修复后的代码虽然顺利执行完毕且 0-Error，但比例关系超出了预设边界（即满足 $ratio > 5$ 或 $ratio < 0.2$ ），代表**业务核心数值发生了非预期的剧烈阶跃退化**。沙箱触发一票否决制，判定该 Patch 为破坏性代码退化，予以拦截并回滚。

7. 后置流插件生态架构 (Plugins Ecosystem)

作为报告的补充延伸，当代码百分百跨越了并列四维门禁且在沙箱中实现了“0-Error、0-Warning、数据无回归”后，修复闭环正式达成。系统会将内存快照刷写回主物理分支，并顺次触发后置流插件生态 (Plugins) 进行代码质量的高级升华：

- **表现层与接入网关 (UI & API Layer)**：借由 VS Code 原生插件 (`extension/`) 中的 `fix.ts` 抓取文件路径，通过基于 FastAPI 搭建的常驻 Web 服务器路由 (`routes/`) 及 Pydantic 强类型契约 (`schemas/`) 维持高频数据长轮询。
- **代码风格自动化审计插件 (Linter Plugin)**：无缝调用系统本地的 `black`、`isort` 或 `ruff` 核心工具链，对大模型生成的 Patch 进行统一的缩进、换行与 PEP 8 样式规范化，抹除 AI 痕迹。
- **安全特征静态扫描插件 (SAST Plugin)**：调度轻量级安全审计引擎 (如 `bandit`)，对合入的代码段进行高危扫描，严防 AI 在修复过程中引入诸如 `eval()`、隐蔽的外部进程拉起或网络挂钩等次生安全漏洞。
- **AI 单测自动补全插件 (Test Generation Tool)**：提取本次修复前后的 AST 差异树 (Diff Matrix)，精准识别故障自愈所牵涉到的方法功能边界，随后自动化针对性生成高覆盖率的 `pytest` 单元测试用例，将修复成果固化入主干仓库，强力防止历史漏洞在未来的代码演进中发生二次复现。